

Progetto di programmazione avanzata e parallela

Parte 1 di 2 - Linguaggio C

Lo scopo di questo progetto è la realizzazione di un interprete per un linguaggio di programmazione stack-based che operi direttamente su tensori (per semplicità solo a uno e due dimensioni, quindi tensori e matrici) e in cui le operazioni siano ottimizzate e parallelizzate (con OpenMP).

Per semplicità chiameremo questo linguaggio **TensorForth**.

Sintassi del linguaggio

Il linguaggio TensorForth è un linguaggio basato su stack (come Forth) in cui un programma è una sequenza di token letti da sinistra verso destra. Ogni token può:

- inserire un valore sullo stack, come nel caso dei tensore o delle stringhe;
- consumare uno o più valori dalla cima dello stack e sostituirli col risultato, come nel caso degli operatori;
- manipolare direttamente lo stack, come nel caso di **d**, **s**, **o** e **D**.

L'ordine dei token è quindi postfixo: prima si inseriscono gli operandi sullo stack, poi si esegue l'operazione che li consuma. Per esempio, il programma

```
[ 1 2 3 ] [ 4 5 6 ] +
```

prima inserisce sullo stack due tensori 1D e poi applica **+**, lasciando sullo stack il tensore risultato.

I token sono separati da spazi o da ritorni a capo. L'implementazione deve segnalare come errore i casi in cui un'operazione non trovi abbastanza elementi sullo stack oppure riceva operandi incompatibili per tipo o dimensioni.

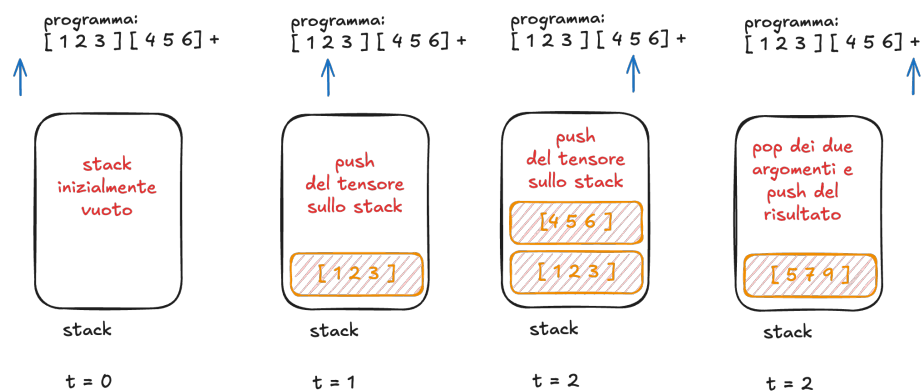


Figure 1: Esempio di esecuzione di un semplice programma

Notazione degli effetti sullo stack Nelle tabelle seguenti l'effetto di ogni operazione sullo stack viene descritto con la notazione:

(elementi-prima -- elementi-dopo)

Gli elementi a sinistra di -- sono quelli che l'operazione si aspetta di trovare sullo stack prima dell'esecuzione, mentre quelli a destra sono quelli presenti dopo l'esecuzione. L'elemento più a destra in ciascun lato della notazione rappresenta la cima dello stack.

Per esempio:

- (b a -- a+b) significa che prima dell'operazione lo stack contiene b sotto a, con a in cima; l'operazione consuma entrambi e lascia in cima il risultato a+b.
- (a -- a a) significa che l'elemento a in cima allo stack viene duplicato.
- (a --) significa che l'elemento a viene consumato e non viene lasciato alcun nuovo valore sullo stack.

I simboli come a, b, m, s e filename sono nomi usati solo per descrivere il ruolo degli operandi; chiaramente non fanno parte della sintassi del linguaggio.

Sintassi dei tensori TensorFlow utilizza unicamente tensori di numeri floating point a precisione singola (float) di una o due dimensioni, quindi vettori 1D o matrici 2D.

Un tensore è rappresentato come:

[n1 n2 n3 ...]

dove n1, n2, n3, etc. sono numero floating point. Lo spazio tra parentesi e tra i diversi numeri è significativo, non devono essere inoltre presenti altri separatori (e.g., virgole). Quindi la seguente notazione è corretta:

[3.4 -8.5 3]

ma le seguenti no:

[3.4 -8.5 3]

[3.4, -8.5, 3]



Si noti come non sia possibile rappresentare in questo modo tensori 2D, per quello è necessaria una operazione di reshape (si vedano operazioni definite successivamente) sul vettore 1D.

Per le operazioni di I/O che richiedono un nome di file, il filename è rappresentato come stringa tra doppi apici, per esempio:

"input.pgm"

"tensor.bin"

Operazioni aritmetiche



Operatore	Effetto sullo stack	Descrizione
+	(b a -- a+b)	somma elemento per elemento
-	(b a -- a-b)	differenza elemento per elemento
*	(b a -- a*b)	prodotto elemento per elemento

In tutti i casi i tensori devono avere le stesse dimensioni

Operazioni di comparazione



Operatore	Effetto sullo stack	Descrizione
<	(b a -- a<b)	confronto elemento per elemento
>	(b a -- a>b)	confronto elemento per elemento
=	(b a -- a=b)	uguaglianza elemento per elemento

In tutti i casi il valore “vero” viene rappresentato da 1.0 e “falso” da 0.0. Inoltre, tutti i tensori devono avere le stesse dimensioni

Operazioni logiche



Operatore	Effetto sullo stack	Descrizione
&	(b a -- a&b)	AND elemento per elemento
\	(b a -- a\ b)	OR elemento per elemento
!	(a -- !a)	negazione elemento per elemento

Il comportamento non è definito se i tensori non sono composti solo da 0.0 e 1.0. Inoltre, tutti i tensori devono avere le stesse dimensioni.

Operazioni di selezione

Operatore	Effetto sullo stack	Descrizione
\$	(b a m -- m?a:b)	seleziona elemento tra a e b in base al valore di m

Si assume che **m** sia un tensore composto solo da 0.0 e 1.0: se il valore è 1.0 viene preso l'elemento corrispondente da a, altrimenti da b. Questo permette di selezionare elementi tra i due tensori in base al tensore maschera m. Chiaramente tutti i tensori devono avere le stesse dimensioni.

Operazioni specifiche per tensori

Operatore	Effetto sullo stack	Descrizione
@	(b a -- a@b)	ritorna il prodotto di matrici tra a e b
.	(b a -- a.b)	ritorna il prodotto interno (dot product) tra a e b
c	(a k -- conv(a,k))	ritorna la convoluzione 2D della matrice a col kernel k con stride 1 e padding di zeri



Per @, i tensori a e b devono essere 2D (quindi matrici) e compatibili come dimensioni. Per ., i tensori a e b devono essere 1D (quindi vettori) di uguale dimensione. Per c, i tensori a e k devono essere 2D (quindi matrici). Il risultato sarà della stessa dimensione di a.

Per un richiamo sul funzionamento della convoluzione si veda l'immagine seguente:

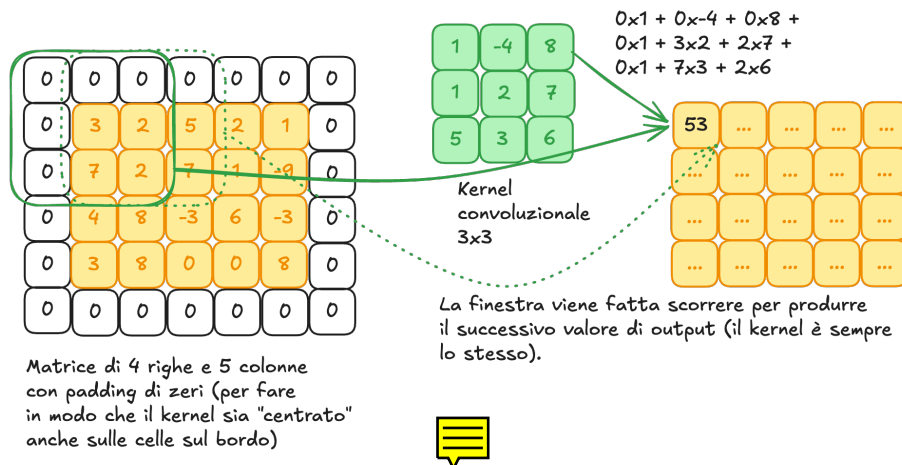


Figure 2: Esempio di convoluzione

Si noti come l'applicazione del kernel convoluzionale per generare l'output sia parallelizzabile dato che ogni valore dell'output è indipendente dagli altri.

Operazioni sulla forma dei tensori

Operatore	Effetto sullo stack	Descrizione
r	(a s -- a') 	ritorna il tensore a con le dimensioni modificate come indicato da s
_	(a -- a') 	(ravel) ritorna la versione “appiattita” (1D) del tensore a
#	(a -- #a)	ritorna un vettore rappresentante le dimensioni di a

Per l'operazione **r** di reshape serve che il numero di elementi in **a** sia lo stesso del prodotto delle dimensioni indicato da **s**. Per esempio se **a** è un vettore di 100 elementi e **s** è [50 2] le dimensioni sono compatibili in quanto la matrice risultante avrà $50 \times 2 = 100$ elementi. La rappresentazione in memoria di **a** e **a'** deve essere la stessa. Anche l'operazione di **_** (ravel) non cambia la rappresentazione in memoria dei valori in **a**. Per **#** (shape) se si ha, per esempio, un tensore **a** di dimensione [100 50] allora **a #** ritornerà [100 50]. Chiaramente il risultato sarà sempre un tensore 1D. Il tensore **s** usato da **r** deve quindi essere un tensore 1D che contiene una o due dimensioni, coerentemente con il vincolo `MAX_DIM = 2`.

Operazioni di generazione di numeri casuali

Operatore	Effetto sullo stack	Descrizione
?	(s - a)	ritorna un tensore delle dimensioni indicate da s composto unicamente da numeri casuali tra 0 e 1

Il tensore **s** deve essere un tensore 1D (come quelli ritornati da **#**) di uno o due elementi. Si noti che, dato che `rand()` non è richiesto sia thread-safe, non è necessario (e non è consigliato) parallelizzare questa operazione.

Operazioni elemento per elemento

Operatore	Effetto sullo stack	Descrizione
R	(a -- relu(a))	tutti gli elementi di a negativi sono sostituiti con 0 ($\text{relu}(x) = \max(0, x)$)
m	(b a -- min(a,b))	ritorna elemento per elemento il minimo tra a e b
M	(b a -- max(a,b))	ritorna elemento per elemento il massimo tra a e b

Per **m** e **M** i tensori devono essere della stessa dimensione.

Operazioni di riduzione

Operatore	Effetto sullo stack	Descrizione
S	(a -- S(a))	Ritorna la somma di tutti i valori in a come tensore 1D di un elemento



Operazioni di filling di tensori

Operatore	Effetto sullo stack	Descrizione
f	(s v -- a)	(fill) crea un tensore di dimensioni date da s riempito coi valori contenuti in v eventualmente ripetuti

Il tensore **s** deve essere un tensore 1D (come quelli ritornati da **#**) di al massimo due elementi. Il tensore risultante conterrà gli elementi di **v** ripetuti. Per esempio `[ 2 3 ] [ 1 2 ] f` creerà la matrice

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 1 & 2 \end{bmatrix}$$



Questa operazione è utile per sostituire il broadcasting (che non è supportato da TensorForth).

Operazioni di utilità

Operatore	Effetto sullo stack	Descrizione
p	(a --)	stampa il contenuto del tensore

Il formato richiesto è nella forma

`Tensor(shape=[shape], data=[data])`

con **shape** la forma del tensore e **data** tutti i dati in esso contenuti. Da utilizzare per il debug. Notare che consuma l'elemento in cima allo stack.

Operazioni di manipolazione dello stack

Operatore	Effetto sullo stack	Descrizione
d	(a -- a a)	(dup) duplica l'elemento in cima allo stack
s	(b a -- a b)	(swap) inverte i due elementi in cima allo stack
o	(b a -- b a b)	(over) duplica il secondo elemento dalla cima dello stack e lo mette in cima allo stack
D	(a --)	(drop) elimina l'elemento in cima allo stack

Le operazioni di dup e over non devono allocare e copiare un nuovo tensore ma aggiornare il numero di riferimenti al tensore. Notate che dovrete deallocare solo quando il numero di references è pari a zero.



I/O operations

Operatore	Effetto sullo stack	Descrizione
(	(filename -- tensor)	legge l'immagine in formato pgm (scala di grigi) binario come tensore normalizzando i valori tra 0 (0) e 1 (255)
)	(a filename --)	scrive un tensore 2D come immagine in formato pgm (scala di grigi) binario
{	(filename -- tensor)	legge il file indicato da filename come tensore (usando mmap e senza copia in memoria)
}	(a filename --)	salva il tensore a nel file filename

Il valore **filename** deve essere una stringa tra doppi apici. Per la scrittura di immagini i valori inferiori a 0 nel tensore originale sono considerati come 0 e quelli superiori a 1 come 1. Nella scrittura dell'immagine i valori in $[0, 1]$ sono poi rimappati in $[0, 255]$ (quindi un tensore di tutti 1 risulterebbe in una immagine bianca).

Formato su disco dei tensori

I tensori devono avere il seguente formato sul disco come inizio del file:

```
struct on_disk_tensor {
    int32_t shape[MAX_DIM];
    int32_t ndim;
    off_t data_offset;
};
```

dove **MAX_DIM** deve essere definito e, per lo scopo del progetto, è fissato a 2. Il valore **data_offset** deve indicare l'offset dall'inizio del file in cui iniziano i dati (che devono essere **float**) facendo in modo che sia l'offset minimo che garantisce che i dati inizino allineati a 64 bytes (not bit!). Dato che **MAX_DIM** è fissato a 2 questo valore sarà, per gli scopi del progetto, pari a 64 e vi saranno quindi alcuni bytes di padding tra **data_offset** e l'inizio dei dati.

Sintassi da linea di comando

Il programma **deve** poter essere chiamato da linea di comando come:

`tensorforth [nome file sorgente]`

dove `[nome file sorgente]` è il nome di un file contenente il sorgente del programma da eseguire.

Programmi di esempio

Sono qui forniti alcuni esempi di programmi con il loro funzionamento

Duplica tensore

`[ 5 5 ] d + p`

Descrizione passo per passo:

1. `[ 5 5 ]` inserisce sullo stack il tensore 1D `[ 5 5 ]`.
2. `d` duplica il tensore in cima allo stack, quindi lo stack contiene due copie logiche di `[ 5 5 ]`.
3. `+` estrae i due tensori dalla cima dello stack e ne calcola la somma elemento per elemento, ottenendo `[ 10 10 ]`.
4. `p` stampa il tensore risultante e lo rimuove dallo stack.

Matrix multiplication

`[ 100 5 ] ? [ 300 ] ? [ 3 100 ] r @ p`

Descrizione passo per passo:

1. `[ 100 5 ]` inserisce sullo stack il tensore forma `[ 100 5 ]`.
2. `?` consuma quel tensore forma e crea una matrice casuale di dimensione `100 x 5`.
3. `[ 300 ]` inserisce sullo stack il tensore 1D `[ 300 ]`.
4. `?` consuma `[ 300 ]` e crea un vettore casuale di 300 elementi.
5. `[ 3 100 ]` inserisce sullo stack il tensore forma `[ 3 100 ]`.
6. `r` estrae il vettore di 300 elementi e il tensore forma `[ 3 100 ]`, quindi fa il reshape del vettore in una matrice `3 x 100`.
7. `@` estrae le due matrici dalla cima dello stack e ne calcola il prodotto di matrici. Le dimensioni sono compatibili perché si moltiplica una matrice `3 x 100` per una matrice `100 x 5`, ottenendo una matrice `3 x 5`.
8. `p` stampa la matrice risultante e la consuma dallo stack.

Blurring di una immagine

`"examples/cray-2.pgm" ([5 5] [0.04] f
c "examples/cray-2-blurred.pgm")`

Descrizione passo per passo:

1. `"examples/cray-2.pgm"` inserisce sullo stack il nome del file immagine di input.

2. `(` legge il file PGM, normalizza i pixel tra 0 e 1 e lascia sullo stack il tensore 2D dell'immagine.
3. `[ 5 5 ]` inserisce sullo stack il tensore forma del kernel di blur.
4. `[ 0.04 ]` inserisce il tensore contenente il valore da ripetere nel kernel.
5. `f` usa forma e valori per creare una matrice 5×5 piena di 0.04; questo corrisponde a un kernel medio uniforme il cui peso totale vale $25 \times 0.04 = 1.0$.
6. `c` applica la convoluzione 2D tra l'immagine e il kernel, producendo l'immagine sfocata.
7. `"examples/cray-2-blurred.pgm"` inserisce sullo stack il nome del file di output.
8. `)` salva il tensore immagine risultante nel file indicato e consuma sia il tensore sia il filename.

Requisiti

- Il programma deve correttamente rilasciare le risorse allocate.
- In caso di errore (e.g., file non apribile) il programma deve segnalare l'errore e uscire con un codice di errore diverso da 0 (quindi non avere segmentation fault).
- Gli errori di stack (stack vuoto o con un numero insufficiente di operandi), di tipo (per esempio filename dove è richiesto un tensore) e di compatibilità delle dimensioni devono essere gestiti esplicitamente e segnalati come errore.
- Ogni funzione rilevante (i.e., non di 2-3 righe) deve essere adeguatamente commentata spiegando che compito svolge, che input richiede e che output genera.
- Ogni file `.c` e `.h` deve contenere all'inizio come commento il proprio nome, cognome e numero di matricola.
- Il Makefile deve essere in grado di compilare l'eseguibile `tensorforth` con la semplice invocazione `make`. In caso di dubbi sull'ambiente si consideri una versione recente di Ubuntu (Linux) come ambiente di test.

Note importanti

- Questo progetto è relativo solamente alla parte in C del corso. Per sostenere la parte di progetto del corso è necessario consegnare sia la parte in C (parte 1) che la parte in Python (parte 2).
- Sebbene appaia ovvio, quando viene chiesto di inserire Nome, Cognome e Matricola dovete inserire il vostro nome, cognome e numero di matricola a non, letteralmente, il testo "Nome", "Cognome" e "Matricola".
- Le istruzioni di consegna saranno fornite con la parte 2 del progetto.
- Sono forniti anche dei programmi di esempio in `examples`. Questi devono essere correttamente eseguiti dal vostro interprete.
- Questo documento verrà aggiornato con chiarificazioni aggiuntive in base ai vostri feedback, controllate se ci sono aggiornamenti!

Last update: 2026-04-28